

```

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# Data set link : https://www.kaggle.com/datasets/moro23/easyvisa-dataset?resource=download&select=EasyVisa.csv

df =
pd.read_csv('/Volumes/study/A_data-science/Python-Code/EDA/EasyVisa_project/EasyVisa.csv')

df.head()

```

	case_id	continent	education_of_employee	has_job_experience	\
0	EZYV01	Asia	High School	N	
1	EZYV02	Asia	Master's	Y	
2	EZYV03	Asia	Bachelor's	N	
3	EZYV04	Asia	Bachelor's	N	
4	EZYV05	Africa	Master's	Y	

	requires_job_training	no_of_employees	yr_of_estab
0	N	14513	2007
1	N	2412	2002
2	Y	44444	2008
3	N	98	1897
4	N	1082	2005

	prevailing_wage	unit_of_wage	full_time_position	case_status
0	592.2029	Hour	Y	Denied
1	83425.6500	Year	Y	Certified
2	122996.8600	Year	Y	Denied
3	83434.0300	Year	Y	Denied
4	149907.3900	Year	Y	Certified

```

df.shape

(25480, 12)

```

Summary of dataset

```
## Summary of dataset , it will show numerical columns only
df.describe()
```

	no_of_employees	yr_of_estab	prevailing_wage
count	25480.000000	25480.000000	25480.000000
mean	5667.043210	1979.409929	74455.814592
std	22877.928848	42.366929	52815.942327
min	-26.000000	1800.000000	2.136700
25%	1022.000000	1976.000000	34015.480000
50%	2109.000000	1997.000000	70308.210000
75%	3504.000000	2005.000000	107735.512500
max	602069.000000	2016.000000	319210.270000

```
# Check null and data types
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 25480 entries, 0 to 25479
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	case_id	25480 non-null	object
1	continent	25480 non-null	object
2	education_of_employee	25480 non-null	object
3	has_job_experience	25480 non-null	object
4	requires_job_training	25480 non-null	object
5	no_of_employees	25480 non-null	int64
6	yr_of_estab	25480 non-null	int64
7	region_of_employment	25480 non-null	object
8	prevailing_wage	25480 non-null	float64
9	unit_of_wage	25480 non-null	object
10	full_time_position	25480 non-null	object
11	case_status	25480 non-null	object

```
dtypes: float64(1), int64(2), object(9)
```

```
memory usage: 2.3+ MB
```

```
df.columns
```

```
Index(['case_id', 'continent', 'education_of_employee',  
      'has_job_experience',  
      'requires_job_training', 'no_of_employees', 'yr_of_estab',  
      'region_of_employment', 'prevailing_wage', 'unit_of_wage',  
      'full_time_position', 'case_status'],  
      dtype='object')
```

Exploring Data

Separate data of Numerical and Categorical columns | you can use any dataset | it is dynamic code .

```
numeric_features = [ feature for feature in df.columns if
df[feature].dtype != '0']
```

```
categorical_features = [ features for feature in df.columns if
df[feature].dtype == '0']
```

```
print('we have {} numerical features :
{}'.format(len(numeric_features), numeric_features))
```

```
print('\nWe have {} Categorical features :
{}'.format(len(categorical_features), categorical_features))
```

```
we have 3 numerical features : ['no_of_employees', 'yr_of_estab',
'prevailing_wage']
```

```
We have 9 Categorical features : [['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage'], ['no_of_employees', 'yr_of_estab',
'prevailing_wage']]
```

define numerical & categorical columns

```
numeric_features = [feature for feature in df.columns if
df[feature].dtype != '0']
```

```
categorical_features = [feature for feature in df.columns if
df[feature].dtype == '0']
```

print columns

```
print('We have {} numerical features :
{}'.format(len(numeric_features), numeric_features))
```

```
print('\nWe have {} categorical features :
{}'.format(len(categorical_features), categorical_features))
```

```
We have 3 numerical features : ['no_of_employees', 'yr_of_estab',
'prevailing_wage']
```

```
We have 9 categorical features : ['case_id', 'continent',
'education_of_employee', 'has_job_experience',
'requires_job_training', 'region_of_employment', 'unit_of_wage',
'full_time_position', 'case_status']
```

```
df.head(2)
```

	case_id	continent	education_of_employee	has_job_experience	\
0	EZYV01	Asia	High School		N
1	EZYV02	Asia	Master's		Y

	requires_job_training	no_of_employees	yr_of_estab
0	West	14513	2007
1	Northeast	2412	2002

	prevailing_wage	unit_of_wage	full_time_position	case_status
0	592.2029	Hour	Y	Denied
1	83425.6500	Year	Y	Certified

len(...): Calculates how many features are in each list..format(...): Replaces the {} placeholders inside the text string with the evaluated values:First {} → gets the count (number of features).Second {} → gets the actual list of column names.\n: Adds a line break (newline) before the second printed line so the output is cleanly spaced out.

```
numeric_features
```

```
['no_of_employees', 'yr_of_estab', 'prevailing_wage']
```

```
f = len(numeric_features)
```

```
f
```

```
3
```

```
df.head(2)
```

	case_id	continent	education_of_employee	has_job_experience	\
0	EZYV01	Asia	High School		N
1	EZYV02	Asia	Master's		Y

	requires_job_training	no_of_employees	yr_of_estab
0	West	14513	2007
1	Northeast	2412	2002

	prevailing_wage	unit_of_wage	full_time_position	case_status
0	592.2029	Hour	Y	Denied
1	83425.6500	Year	Y	Certified

```
# values counts
```

```
df['continent'].value_counts()
```

```

continent
Asia          16861
Europe        3732
North America 3292
South America 852
Africa         551
Oceania        192
Name: count, dtype: int64

# proportion of count data on categorical columns

'''for col in categorical_features:
    print(df[col].value_counts(normalize = True) * 100)
    print('-----')'''

for col in categorical_features:
    print(df[col].value_counts(normalize=True) * 100)
    print('-----')

case_id
EZYV01      0.003925
EZYV16995   0.003925
EZYV16993   0.003925
EZYV16992   0.003925
EZYV16991   0.003925
...
EZYV8492    0.003925
EZYV8491    0.003925
EZYV8490    0.003925
EZYV8489    0.003925
EZYV25480   0.003925
Name: proportion, Length: 25480, dtype: float64
-----
continent
Asia          66.173469
Europe        14.646782
North America 12.919937
South America  3.343799
Africa         2.162480
Oceania        0.753532
Name: proportion, dtype: float64
-----
education_of_employee
Bachelor's    40.164835
Master's      37.810047
High School   13.422292
Doctorate     8.602826
Name: proportion, dtype: float64
-----

```

```
has_job_experience
Y      58.092622
N      41.907378
Name: proportion, dtype: float64
-----
```

```
requires_job_training
N      88.402669
Y      11.597331
Name: proportion, dtype: float64
-----
```

```
region_of_employment
Northeast    28.237834
South        27.539246
West         25.847724
Midwest      16.903454
Island        1.471743
Name: proportion, dtype: float64
-----
```

```
unit_of_wage
Year        90.117739
Hour         8.465463
Week         1.067504
Month        0.349294
Name: proportion, dtype: float64
-----
```

```
full_time_position
Y      89.375981
N      10.624019
Name: proportion, dtype: float64
-----
```

```
case_status
Certified    66.789639
Denied       33.210361
Name: proportion, dtype: float64
-----
```

```
# proportion of count data on categorical columns
```

```
for col in numeric_features:
    print(df[col].value_counts(normalize = True) * 100)
    print('-----')
```

```
no_of_employees
183      0.070644
854      0.062794
724      0.062794
766      0.058870
1476     0.058870
...
5876     0.003925
```

```
5536      0.003925
47866     0.003925
4700      0.003925
40224     0.003925
```

```
Name: proportion, Length: 7105, dtype: float64
```

```
-----
yr_of_estab
```

```
1998      4.450549
2005      4.124804
2001      3.991366
2007      3.901099
1999      3.414443
```

```
...
1842      0.023548
1846      0.015699
1822      0.015699
1810      0.011774
1824      0.007849
```

```
Name: proportion, Length: 199, dtype: float64
```

```
-----
prevailing_wage
```

```
82560.2800    0.007849
122.6500      0.007849
60948.1500    0.007849
64357.5800    0.007849
108.1200      0.007849
```

```
...
25713.9800    0.003925
101656.6400   0.003925
65665.5500    0.003925
50.8813       0.003925
70876.9100    0.003925
```

```
Name: proportion, Length: 25454, dtype: float64
```

Insights

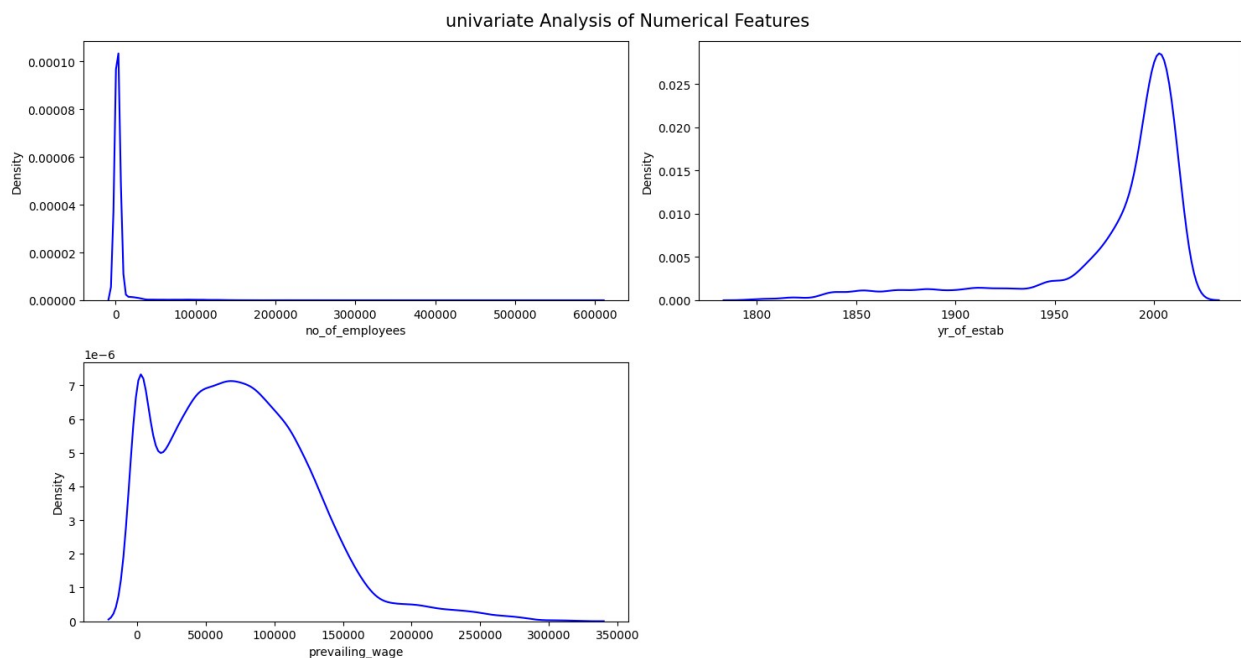
- Case_id have unique values for each column which can be dropped as it is of no importance
- continent column is highly towards asia, hence we can combine other categories to form a single category.
- unit_of_wage seems to be an important column as most of them are yearly contracts.

Univariate Analysis

Numerical Features

```
plt.figure(figsize = (15, 8))
plt.suptitle("univariate Analysis of Numerical Features", fontsize
=15, fontweight=3)

for i in range(0, len(numeric_features)):
    plt.subplot(2,2, i+1)
    sns.kdeplot(x=df[numeric_features[i]], color = 'blue')
    plt.xlabel(numeric_features[i])
    plt.tight_layout()
```



** Insights**

- `no_of_employees` column looks more skewed but it may be because of outliers.
- `yr_of_estab` is a discrete feature.
- `prevailing_wage` is a right-skewed distribution.

```
df.head()
```

	case_id	continent	education_of_employee	has_job_experience	\
0	EZYV01	Asia	High School	N	
1	EZYV02	Asia	Master's	Y	
2	EZYV03	Asia	Bachelor's	N	
3	EZYV04	Asia	Bachelor's	N	
4	EZYV05	Africa	Master's	Y	

	requires_job_training	no_of_employees	yr_of_estab
region_of_employment \			
0	N	14513	2007
West			
1	N	2412	2002
Northeast			
2	Y	44444	2008
West			
3	N	98	1897
West			
4	N	1082	2005
South			

	prevailing_wage	unit_of_wage	full_time_position	case_status
0	592.2029	Hour	Y	Denied
1	83425.6500	Year	Y	Certified
2	122996.8600	Year	Y	Denied
3	83434.0300	Year	Y	Denied
4	149907.3900	Year	Y	Certified

Categorical Features

```
'''# categorical columns
#categorical_features

#categorical_features.remove('case_id')
plt.figure(figsize=(15, 8))
plt.suptitle('Univariate Analysis of categorical Features', fontsize
=20, fontweight = 3)

for i in range(0, len(categorical_features)):
    plt.subplot(3, 3, i+1)
    sns.countplot(x = df[categorical_features[i]])
    plt.xlabel(categorical_features[i])
    plt.tight_layout()'''

"# categorical columns\n#categorical_features\n\
n#categorical_features.remove('case_id')\nplt.figure(figsize=(15,
8))\nplt.suptitle('Univariate Analysis of categorical Features',
fontsize =20, fontweight = 3)\n\nfor i in range(0,
len(categorical_features)):\n    plt.subplot(3, 3, i+1)\n
sns.countplot(x = df[categorical_features[i]])\n
plt.xlabel(categorical_features[i])\n    plt.tight_layout()"

# categorical columns
categorical_features.remove('case_id')
plt.figure(figsize=(15, 8))
plt.suptitle('Univariate Analysis of Categorical Features',
```

```

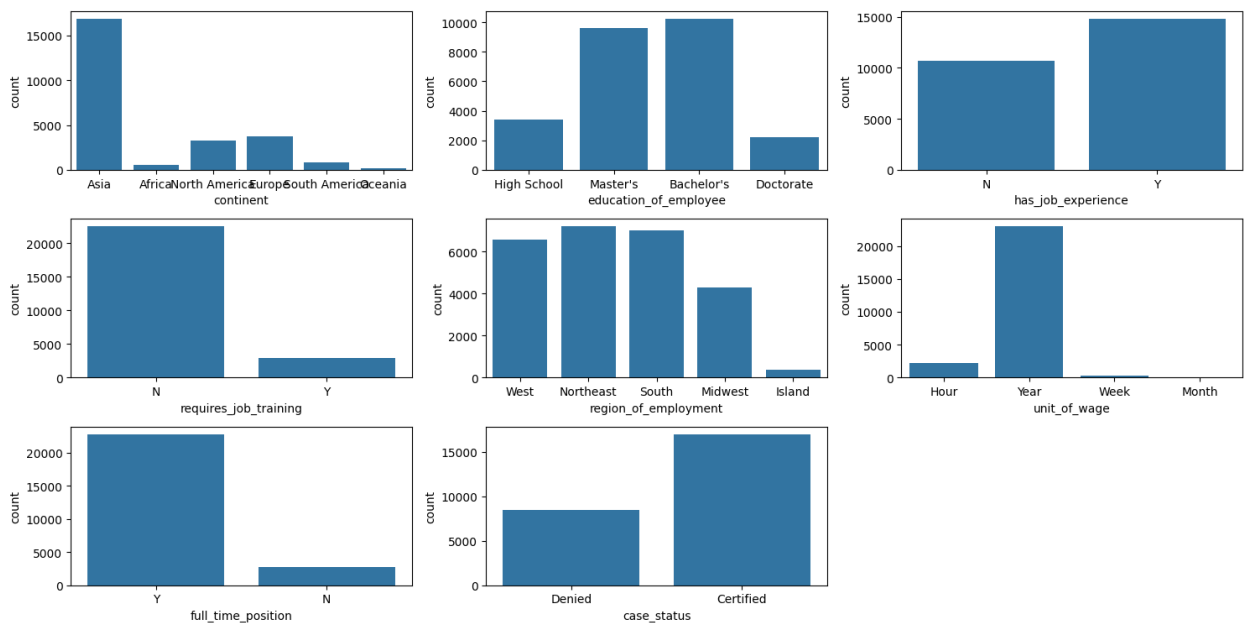
fontsize=20, fontweight='bold', alpha=0.8, y=1.)

for i in range(0, len(categorical_features)):
    plt.subplot(3, 3, i+1)
    sns.countplot(x=df[categorical_features[i]])
    plt.xlabel(categorical_features[i])
    plt.tight_layout()

# save plot
# plt.savefig('./images/Univariate_Cat.png')

```

Univariate Analysis of Categorical Features



insights

- requires_job_training, unit_of_wage, full_time_position and continents columns have single categorical doming = in rest of the columns are balanced

Multivariate Analysis

```

# numeric_features
numeric_features
['no_of_employees', 'yr_of_estab', 'prevailing_wage']
len(df['no_of_employees'].unique())
7105

```

```

discrete_features = [feature for feature in numeric_features if
len(df[feature].unique())<=25]

continuous_features = [feature for feature in numeric_features if
len(df[feature].unique()) > 25]

print('we have {} discrete features : {}'.format(len(discrete_features),discrete_features))

print('we have {} continuours _features : {}'.format(len(continuous_features), continuous_features))

we have 0 discrete features : []
we have 3 continuours _features : ['no_of_employees', 'yr_of_estab',
'prevailing_wage']

discrete_features

[]

continuous_features

['no_of_employees', 'yr_of_estab', 'prevailing_wage']

```

Check multicollinearity in Numerical features

```
df.head(2)
```

	case_id	continent	education_of_employee	has_job_experience	\
0	EZYV01	Asia	High School		N
1	EZYV02	Asia	Master's		Y

	requires_job_training	no_of_employees	yr_of_estab
region_of_employment \			
0	N	14513	2007
West			
1	N	2412	2002
Northeast			

	prevailing_wage	unit_of_wage	full_time_position	case_status
0	592.2029	Hour	Y	Denied
1	83425.6500	Year	Y	Certified

```
df.corr(numeric_only = True)
```

	no_of_employees	yr_of_estab	prevailing_wage
no_of_employees	1.000000	-0.017770	-0.009523
yr_of_estab	-0.017770	1.000000	0.012342
prevailing_wage	-0.009523	0.012342	1.000000

```
# only categorical columns

'''from scipy.stats import chi2_contingency

chi2_test = []

for feature in categorical_features:
    if chi2_contingency(pd.crosstab(df['case_status'], df[feature]))
[1] < 0.05:
        chi2_test.append('Reject null hypothesis')
    else :
        chi2_test.append('Fail to Reject Null Hypothesis')

result = pd.DataFrame(data = [categorical_features, chi2_test]).T
result.columns = ['column', 'Hypothesis Result']'''

"from scipy.stats import chi2_contingency\n\nchi2_test = []\n\nfor
feature in categorical_features:\n    if
chi2_contingency(pd.crosstab(df['case_status'], df[feature]))[1] <
0.05:\n        chi2_test.append('Reject null hypothesis')\n    else :\n
n        chi2_test.append('Fail to Reject Null Hypothesis')\n\nresult
= pd.DataFrame(data = [categorical_features, chi2_test]).T\n
nresult.columns = ['column', 'Hypothesis Result']"

from scipy.stats import chi2_contingency
chi2_test = []
for feature in categorical_features:
    if chi2_contingency(pd.crosstab(df['case_status'], df[feature]))
[1] < 0.05:
        chi2_test.append('Reject Null Hypothesis')
    else:
        chi2_test.append('Fail to Reject Null Hypothesis')
result = pd.DataFrame(data=[categorical_features, chi2_test]).T
result.columns = ['Column', 'Hypothesis Result']
result
```

	Column	Hypothesis Result
0	continent	Reject Null Hypothesis
1	education_of_employee	Reject Null Hypothesis
2	has_job_experience	Reject Null Hypothesis
3	requires_job_training	Fail to Reject Null Hypothesis
4	region_of_employment	Reject Null Hypothesis
5	unit_of_wage	Reject Null Hypothesis
6	full_time_position	Reject Null Hypothesis
7	case_status	Reject Null Hypothesis

Checking Null values

```
df.isnull().sum()
```

```

case_id          0
continent        0
education_of_employee  0
has_job_experience  0
requires_job_training  0
no_of_employees  0
yr_of_estab      0
region_of_employment  0
prevailing_wage  0
unit_of_wage     0
full_time_position  0
case_status      0
dtype: int64

```

```

"""continues_features = [feature for feature in numeric_features if
len(df[df[feature].unique()) > 10]

```

```

print("Num of continues features : ", continues_features)"""

```

```

continues_features=[feature for feature in numeric_features if
len(df[feature].unique())>=10]
print('Num of continues features :',continues_features)

```

```

Num of continues features : ['no_of_employees', 'yr_of_estab',
'prevailing_wage']

```

```

clr1 = ['#1E90FF', '#DC143C']
fig, ax = plt.subplots(3, 2, figsize=(10,12))
fig.suptitle('Distribution of Numerical Features By Case Status',
color='#3C3744',
                fontsize=20, fontweight='bold', ha='center')
for i, col in enumerate(continues_features):
    sns.boxplot(data=df, x='case_status', y=col, palette=clr1,
ax=ax[i,0])
    ax[i,0].set_title(f'Boxplot of {col}', fontsize=12)
    sns.histplot(data=df, x=col, hue='case_status', bins=20, kde=True,
                multiple='stack', palette=clr1, ax=ax[i,1])
    ax[i,1].set_title(f'Histogram of {col}', fontsize=14)
fig.tight_layout()
fig.subplots_adjust(top=0.90)

```

```

plt.show()
# plt.savefig('images/multivariate_num.png')

```

```

/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/
ipykernel_32543/1037160477.py:6: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

``legend=False`` for the same effect.

```
sns.boxplot(data=df, x='case_status', y=col, palette=clr1,  
ax=ax[i,0])  
/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/ipykernel_32543/10371  
60477.py:6: FutureWarning:
```

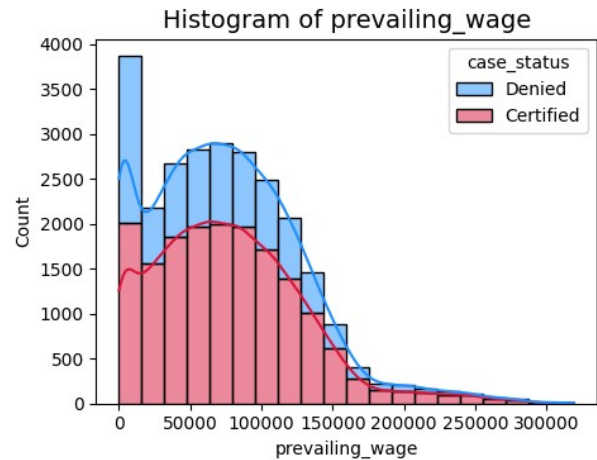
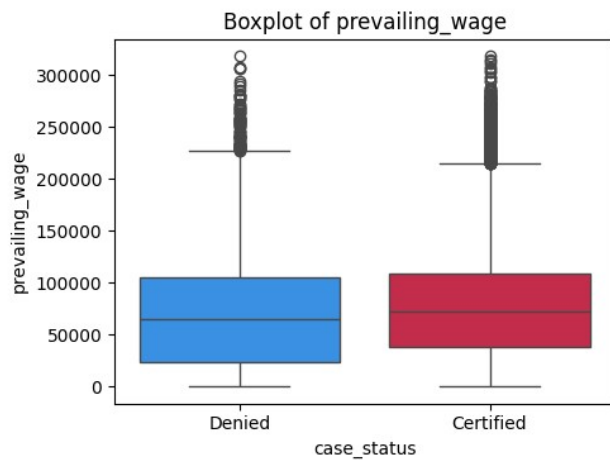
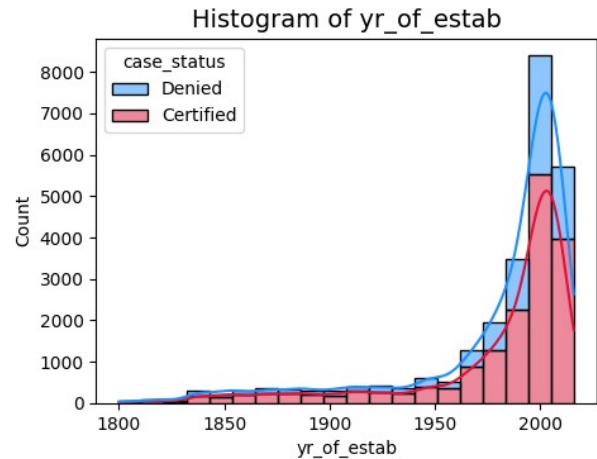
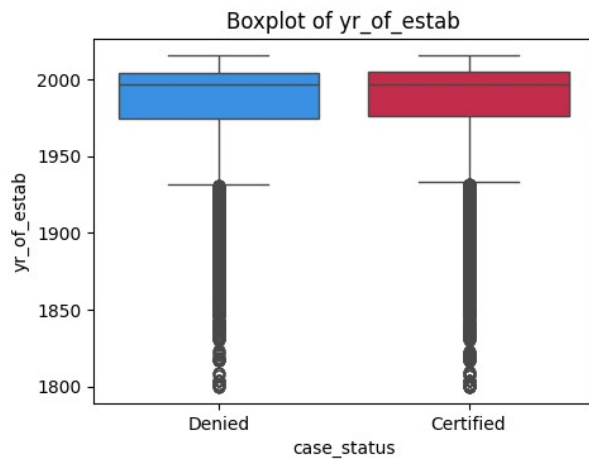
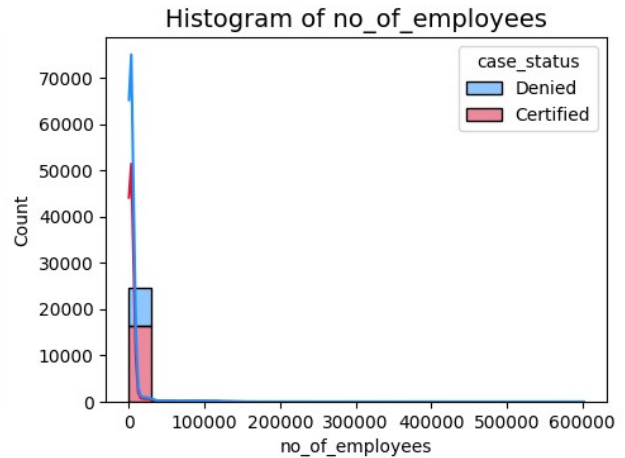
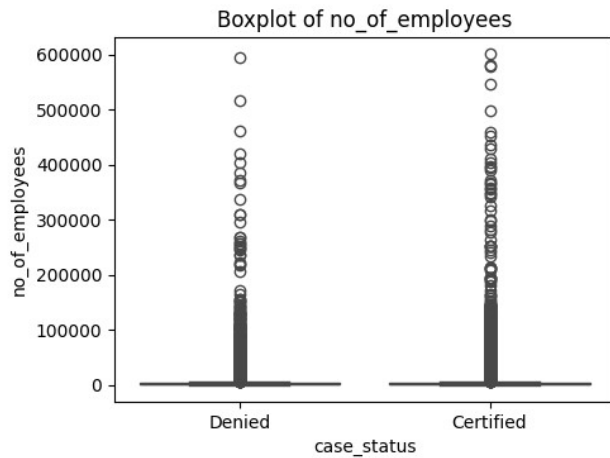
Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0. Assign the ``x`` variable to ``hue`` and set ``legend=False`` for the same effect.

```
sns.boxplot(data=df, x='case_status', y=col, palette=clr1,  
ax=ax[i,0])  
/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/ipykernel_32543/10371  
60477.py:6: FutureWarning:
```

Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0. Assign the ``x`` variable to ``hue`` and set ``legend=False`` for the same effect.

```
sns.boxplot(data=df, x='case_status', y=col, palette=clr1,  
ax=ax[i,0])
```

Distribution of Numerical Features By Case Status



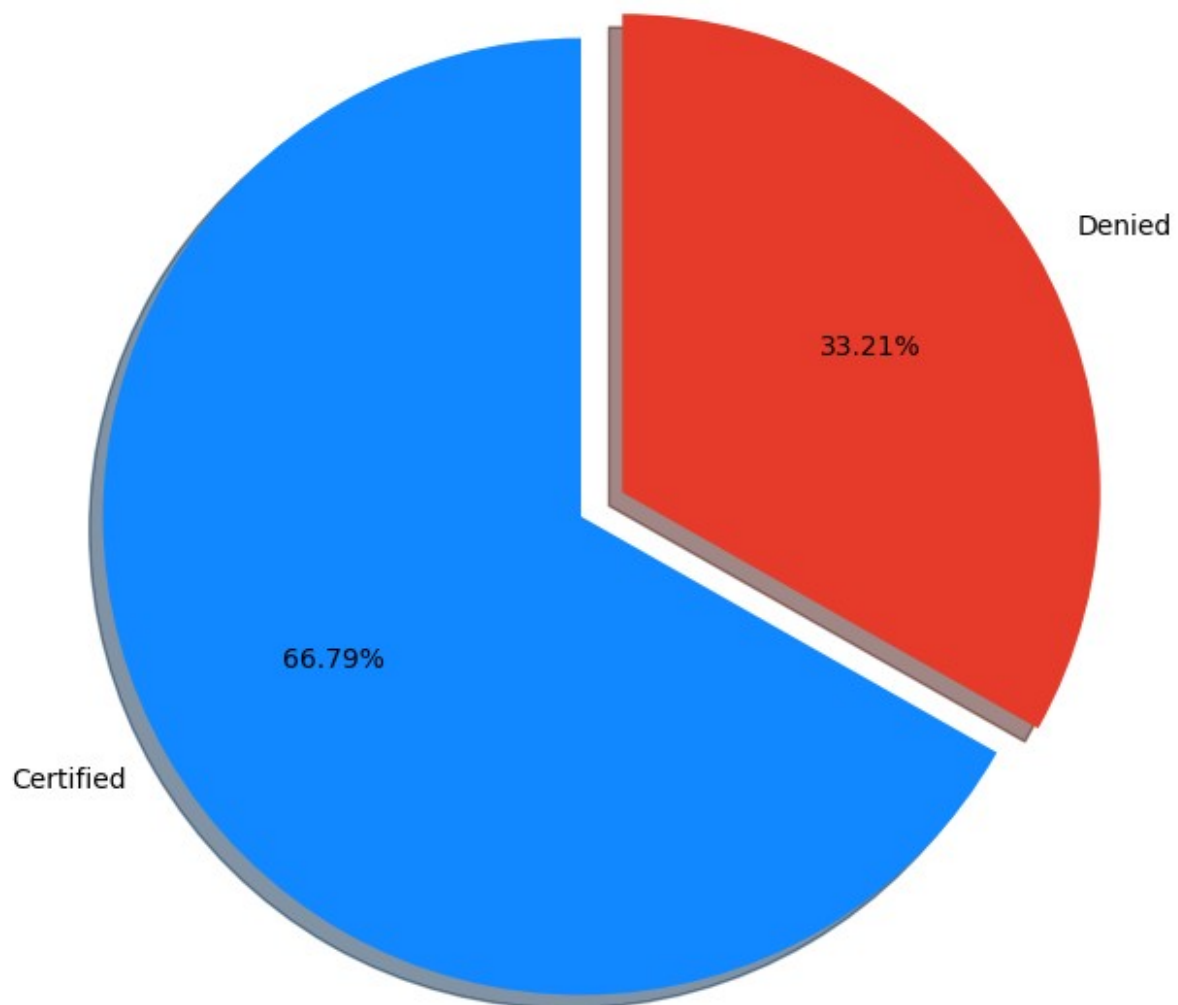
Initial Analysis Report

- No of Employees has many outliers which can be Handled in Feature Engineering and `no_of_employees` is Right Skewed.
- `yr_of_estab` is left skewed and some outliers below the lower bound of Box plot.
- `prevailing_wage` is right skewed with outliers above upper bound of box plot.
- There are No missing values in the dataset.
- The `case_id` column can be deleted because each row has unique values.
- The `case_status` column is the target to predict.
- In the Categorical column, features can be made Binary numerical in feature Encoding

Visualize the Target Feature

```
percentage = df.case_status.value_counts(normalize=True)*100
labels = ["Certified", "Denied"]

# Plot PieChart with Plotly library
fig, ax = plt.subplots(figsize=(15, 8))
explode = (0, 0.1)
colors = ['#1188ff', '#e63a2a']
ax.pie(percentage, labels = labels, startangle = 90,
        autopct='%1.2f%%', explode=explode, shadow=True, colors=colors)
plt.show()
```

- From the chart it is seen that the Target Variable is Imbalanced **What is imbalanced data?**

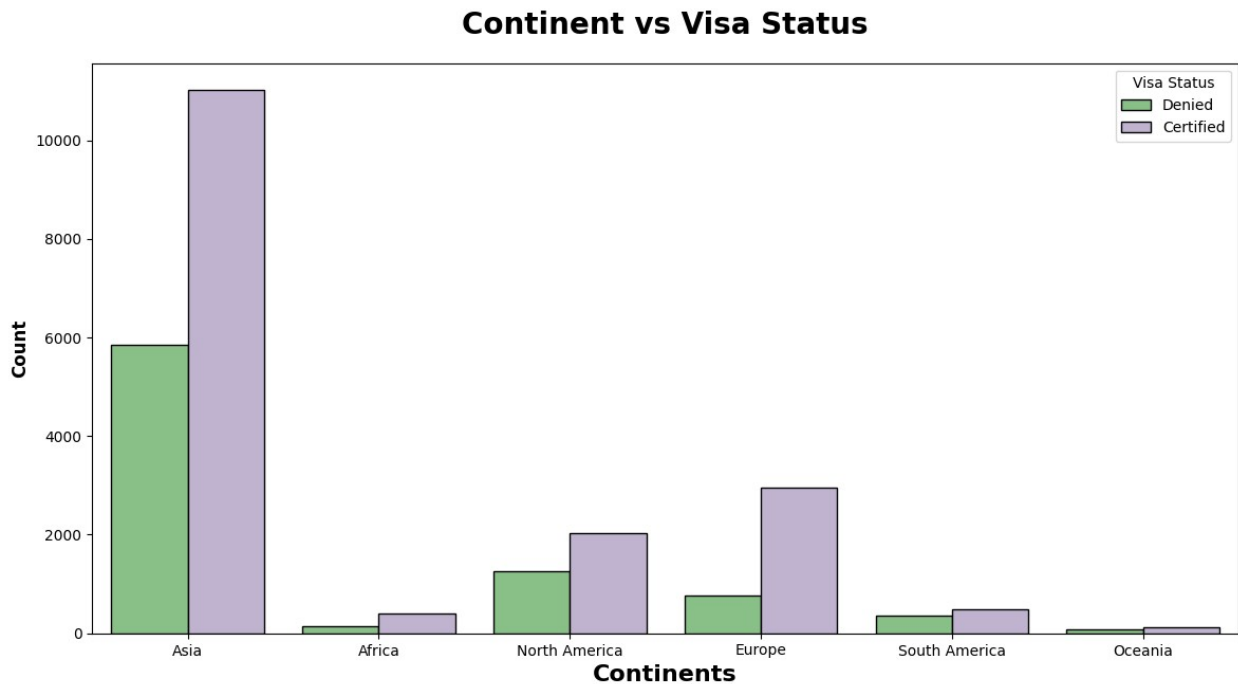
Imbalanced data are types of data where the target class has an uneven distribution of observations, i.e Here Denied value has more count than the Certified value of the dataset.

Does applicant Continent has any impact on Visa status?

```
#group data by continent and their count of case_status
df.groupby('continent')
['case_status'].value_counts(normalize=True).to_frame()*100
```

continent	case_status	proportion
Africa	Certified	72.050817
	Denied	27.949183
Asia	Certified	65.310480
	Denied	34.689520
Europe	Certified	79.233655
	Denied	20.766345
North America	Certified	61.877278
	Denied	38.122722
Oceania	Certified	63.541667
	Denied	36.458333
South America	Certified	57.863850
	Denied	42.136150

```
plt.subplots(figsize=(14,7))
sns.countplot(x="continent",hue="case_status", data=df, ec =
"black",palette="Accent")
plt.title("Continent vs Visa Status", weight="bold",fontsize=20,
pad=20)
plt.ylabel("Count", weight="bold", fontsize=12)
plt.xlabel("Continents", weight="bold", fontsize=16)
plt.legend(title="Visa Status", fancybox=True)
plt.show()
```



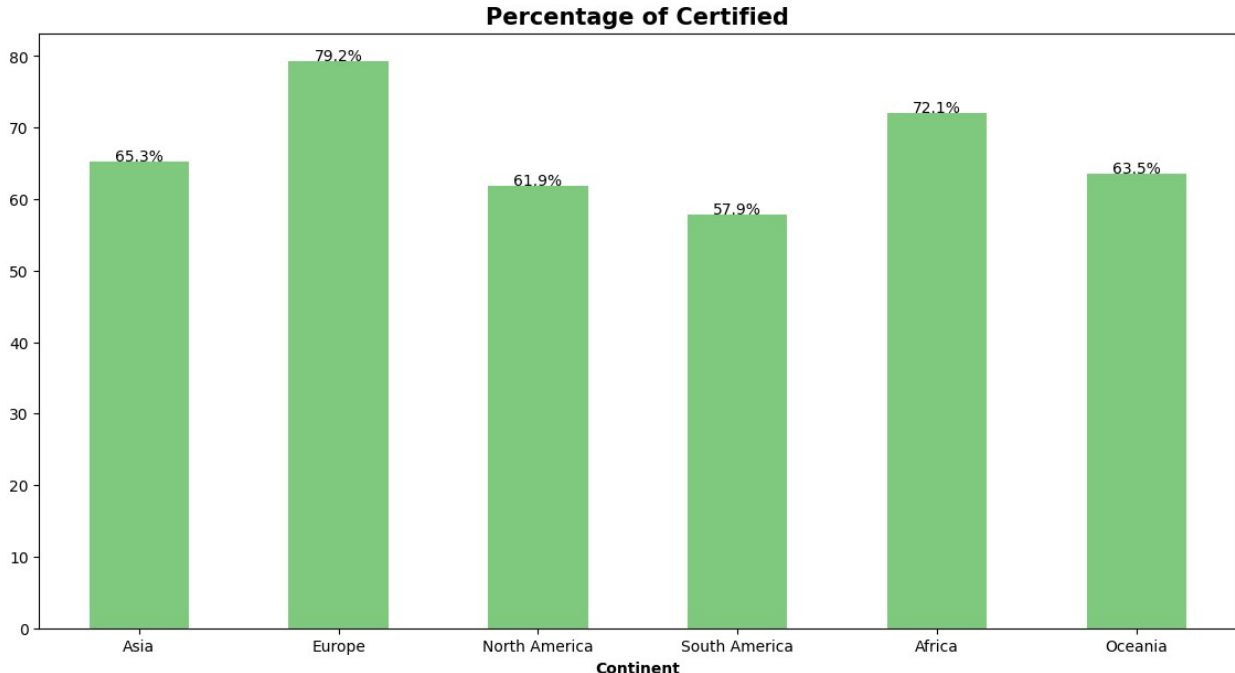
```
df2 = df.copy()
```

```
plt.figure(figsize=[14,7])

(100*df2[df2["case_status"].isin(['Certified'])]
['continent'].value_counts()/df2['continent'].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of Certified" , fontsize = 15, fontweight
='bold' )
order1 = (100*df2[df2["case_status"].isin(['Certified'])]
['continent'].value_counts()/df2['continent'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('Continent' , fontweight = 'bold')
plt.xticks(rotation=0)
plt.show()

/var/folders/9f/00d3gq1s0w3_0m39lmyx76400000gn/T/
ipykernel_32543/4062120739.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    count = order1[n]
```



Report:

- As per the Chart Asia applicants applied more than other continents.
- 43% of Certified applications are from Asia.

- This is followed by Europe with 11% of Certified applications.
- Highest chance of getting certified if you are from Europe and followed by Africa

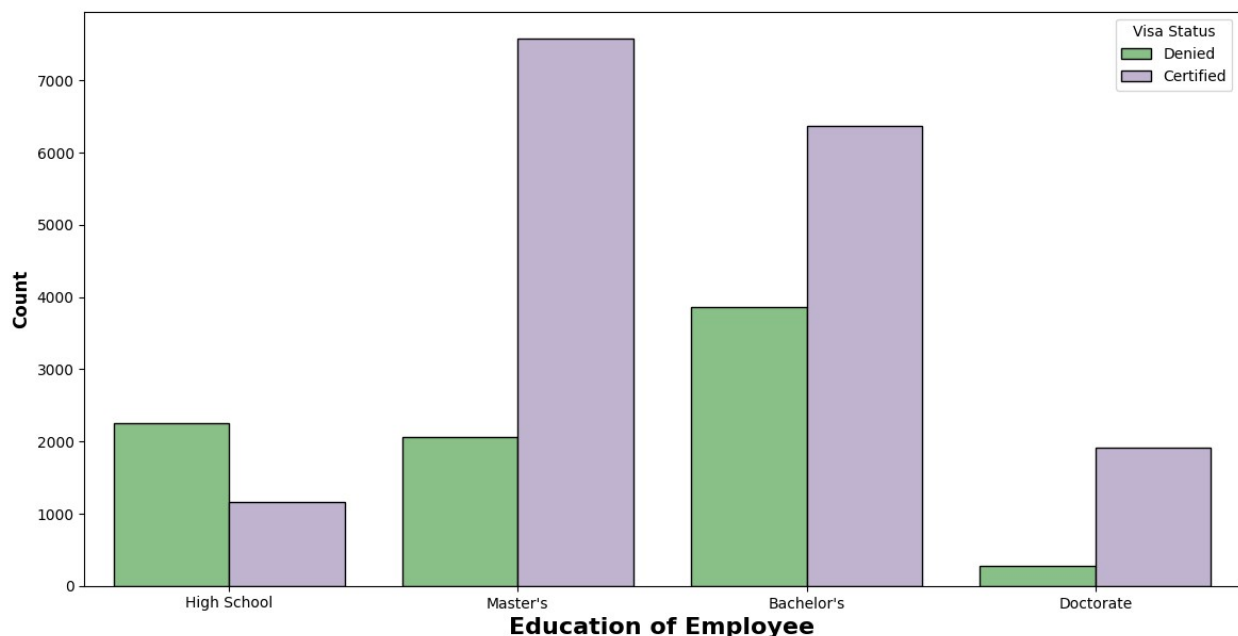
Does applicant education has any impact on Visa status ?

```
#group data by Education and their count of case_status
df.groupby('education_of_employee')
['case_status'].value_counts(normalize=True).to_frame()*100
```

education_of_employee	case_status	proportion
Bachelor's	Certified	62.214188
	Denied	37.785812
Doctorate	Certified	87.226277
	Denied	12.773723
High School	Denied	65.964912
	Certified	34.035088
Master's	Certified	78.627777
	Denied	21.372223

```
plt.subplots(figsize=(14,7))
sns.countplot(x="education_of_employee",hue="case_status", data=df, ec
= "black",palette="Accent")
plt.title("Education of Employee vs Visa Status",
weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=12)
plt.xlabel("Education of Employee", weight="bold", fontsize=16)
plt.legend(title="Visa Status", fancybox=True)
plt.show()
```

Education of Employee vs Visa Status

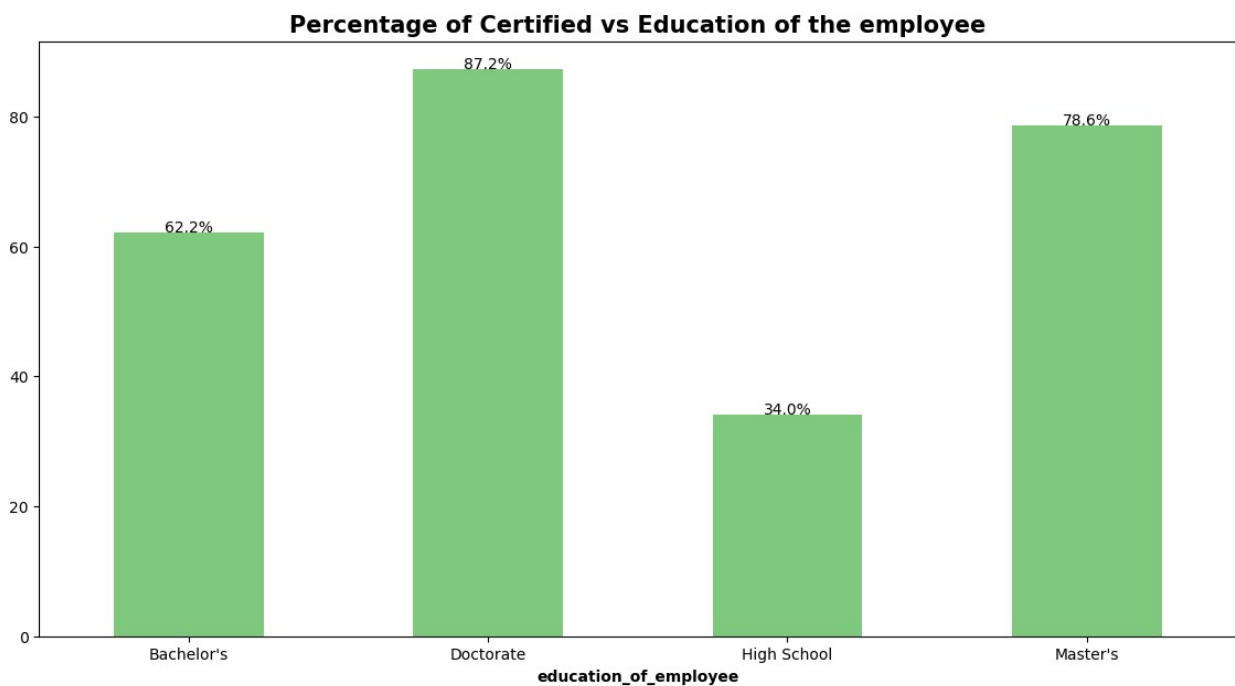


```
plt.figure(figsize=[14,7])

(100*df2[df2["case_status"].isin(['Certified'])]
['education_of_employee'].value_counts()/
df2['education_of_employee'].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of Certified vs Education of the employee" ,
    fontsize = 15, fontweight = 'bold' )
order1 = (100*df2[df2["case_status"].isin(['Certified'])]
['education_of_employee'].value_counts()/
df2['education_of_employee'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('education_of_employee' , fontweight = 'bold')
plt.xticks(rotation=0)
plt.show()

/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/
ipykernel_32543/1356139597.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    count = order1[n]
```



Insights

- education status has high impact
- Doctorate and Master's graduates have higher chance of being accepted than the others.

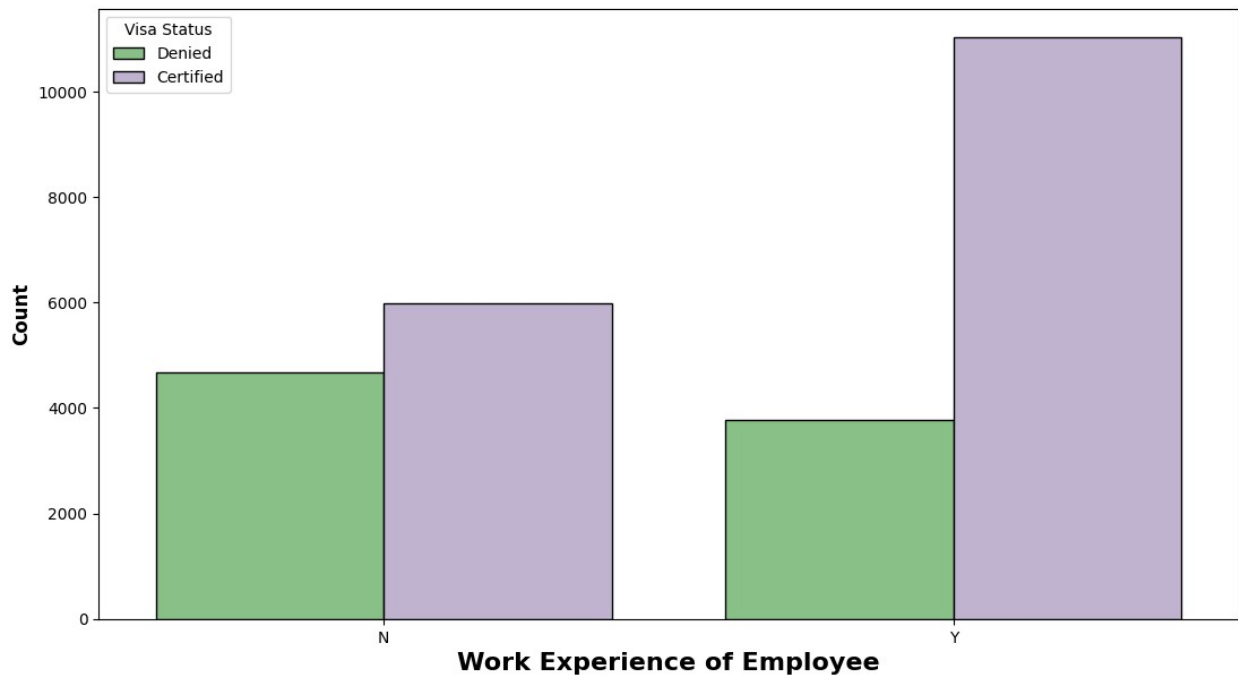
Does applicant's previous work experience has any impact on Visa Status?

```
#group data by has_job_experience and count case_status
df.groupby('has_job_experience')
['case_status'].value_counts(normalize=True).to_frame()*100
```

has_job_experience	case_status	proportion
N	Certified	56.134108
	Denied	43.865892
Y	Certified	74.476422
	Denied	25.523578

```
plt.subplots(figsize=(13,7))
sns.countplot(x="has_job_experience",hue="case_status", data=df, ec=
"black",palette="Accent")
plt.title("Previous Work Experience of Employee vs Visa Status",
weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=12)
plt.xlabel("Work Experience of Employee", weight="bold", fontsize=16)
plt.legend(title="Visa Status", fancybox=True)
plt.show()
```

Previous Work Experience of Employee vs Visa Status

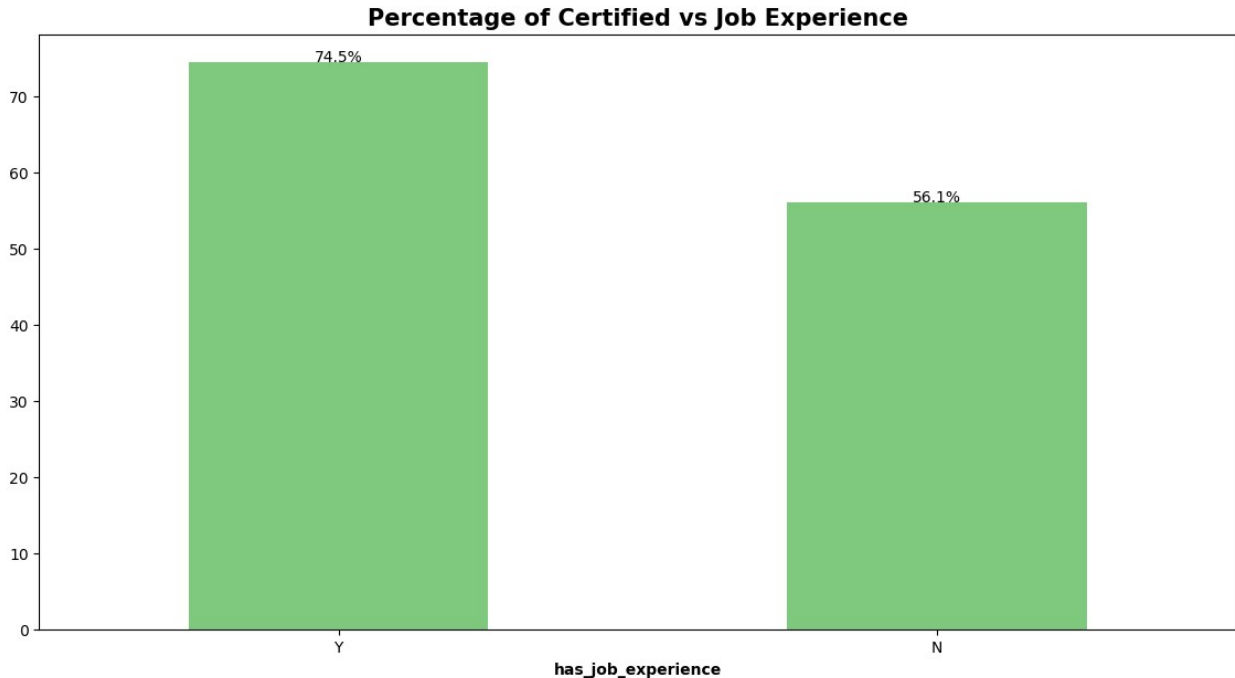


```
plt.figure(figsize=[14,7])

(100*df2[df2["case_status"].isin(['Certified'])]
['has_job_experience'].value_counts()/
df2['has_job_experience'].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of Certified vs Job Experience" , fontsize = 15,
fontweight = 'bold' )
order1 = (100*df2[df2["case_status"].isin(['Certified'])]
['has_job_experience'].value_counts()/
df2['has_job_experience'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('has_job_experience' , fontweight = 'bold')
plt.xticks(rotation=0)
plt.show()

/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/
ipykernel_32543/679819683.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    count = order1[n]
```



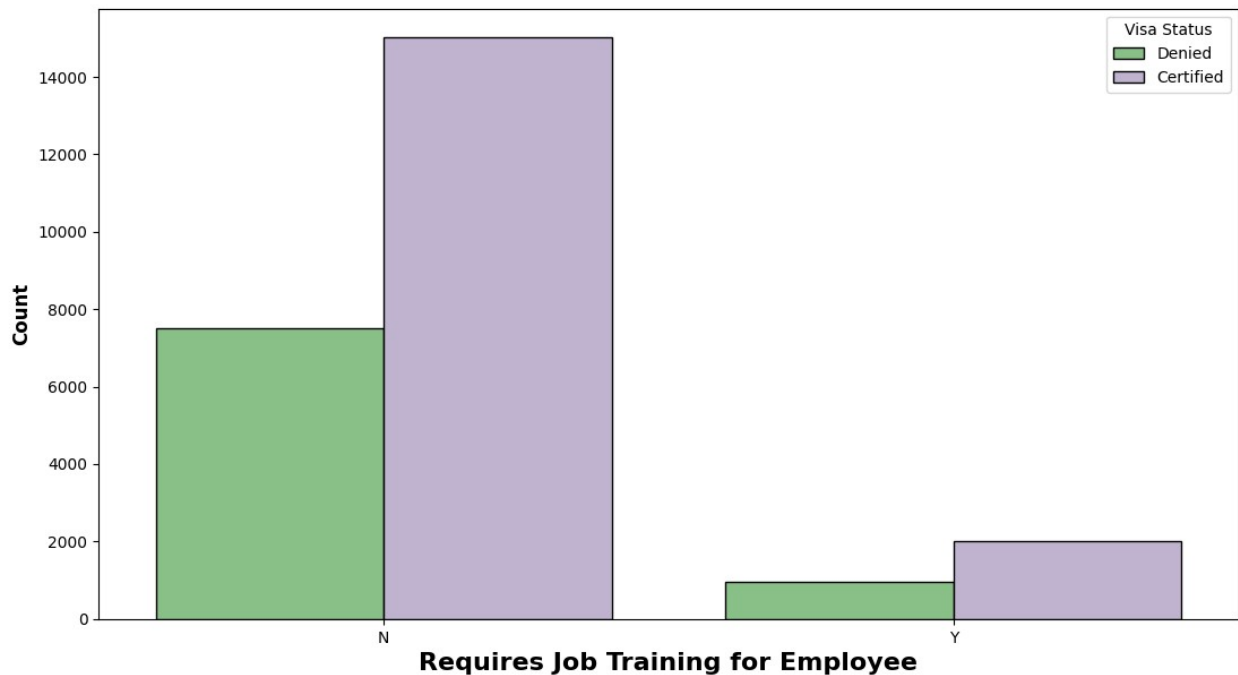
If the Employee requires job training, does it make impact on visa status?

```
#group data by requires_job_training and count case_status
df.groupby('requires_job_training')
['case_status'].value_counts(normalize=True).to_frame()*100
```

		proportion
requires_job_training	case_status	
N	Certified	66.645949
	Denied	33.354051
Y	Certified	67.884941
	Denied	32.115059

```
plt.subplots(figsize=(13,7))
sns.countplot(x="requires_job_training",hue="case_status", data=df,
ec= "black",palette="Accent")
plt.title("Requires job training for Employee vs Visa Status",
weight="bold",fontsize=20, pad=20)
plt.ylabel("Count", weight="bold", fontsize=12)
plt.xlabel("Requires Job Training for Employee", weight="bold",
fontsize=16)
plt.legend(title="Visa Status", fancybox=True)
plt.show()
```


Requires job training for Employee vs Visa Status



```
plt.figure(figsize=[14,7])

(100*df2[df2["case_status"].isin(['Certified'])]
['requires_job_training'].value_counts()/
df2['requires_job_training'].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of Certified vs Required training" , fontsize =
15, fontweight = 'bold' )
order1 = (100*df2[df2["case_status"].isin(['Certified'])]
['requires_job_training'].value_counts()/
df2['requires_job_training'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('requires_job_training' , fontweight = 'bold')
plt.xticks(rotation=0)
plt.show()

/var/folders/9f/00d3gq1s0w3_0m391myx76400000gn/T/
ipykernel_32543/88884806.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    count = order1[n]
```



Wages and it's impact on Visa status

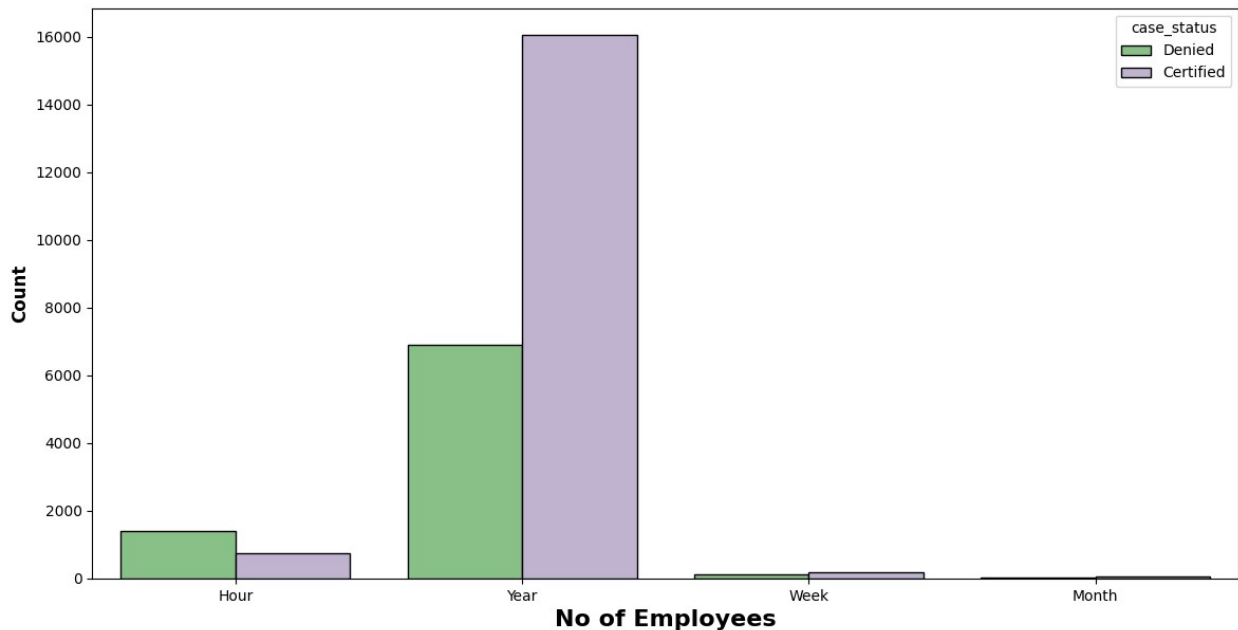
```
#group data by unit_of_wage and count the case_status
df.groupby('unit_of_wage')['case_status'].value_counts(normalize=True)
```

unit_of_wage	case_status	proportion
Hour	Denied	0.653686
	Certified	0.346314
Month	Certified	0.617978
	Denied	0.382022
Week	Certified	0.621324
	Denied	0.378676
Year	Certified	0.698850
	Denied	0.301150

Name: proportion, dtype: float64

```
plt.subplots(figsize=(14,7))
sns.countplot(x="unit_of_wage",hue="case_status", data=df,ec='k',
palette='Accent')
plt.title("No of Employees in company vs Visa Status",
weight="bold",fontsize=20, pad=20)
plt.xlabel("No of Employees", weight="bold", fontsize=16)
plt.ylabel("Count", weight="bold", fontsize=12)
plt.show()
```

No of Employees in company vs Visa Status

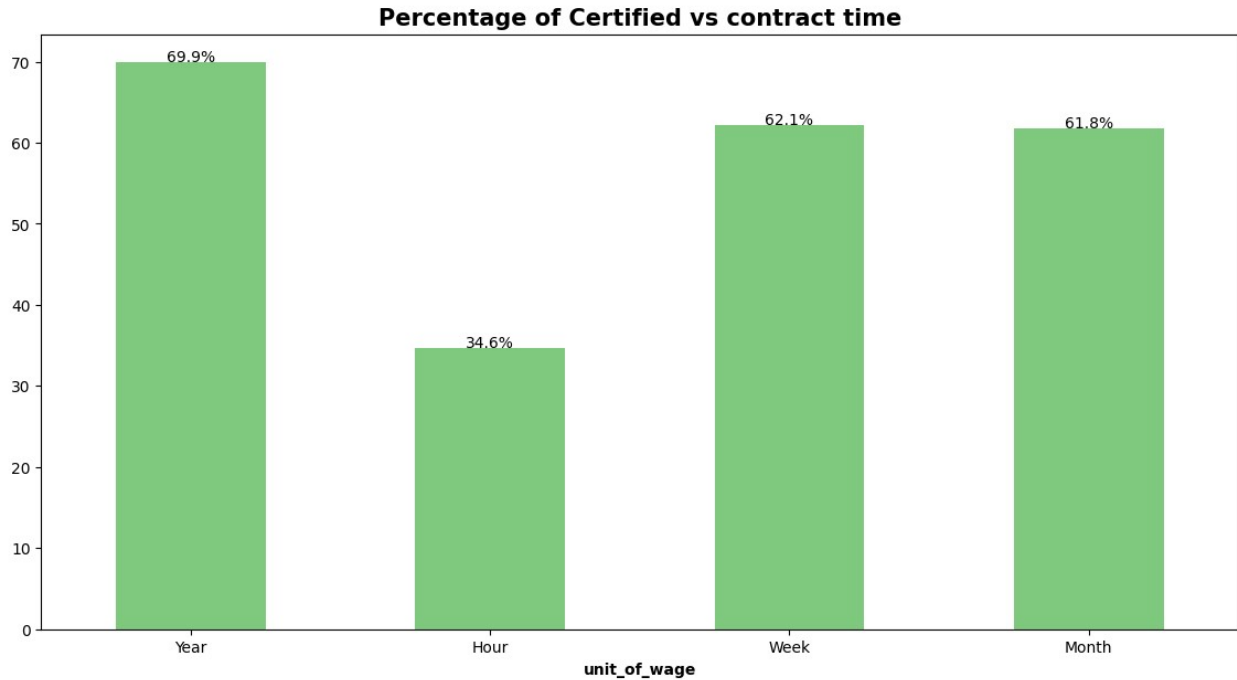


```
plt.figure(figsize=[14,7])

(100*df2[df2["case_status"].isin(['Certified'])]
['unit_of_wage'].value_counts()/
df2['unit_of_wage'].value_counts()).plot(
    kind='bar',stacked=True , colormap='Accent')
plt.title("Percentage of Certified vs contract time" , fontsize = 15,
fontweight = 'bold' )
order1 = (100*df2[df2["case_status"].isin(['Certified'])]
['unit_of_wage'].value_counts()/df2['unit_of_wage'].value_counts())
for n in range(order1.shape[0]):
    count = order1[n]
    strt='{:0.1f}%'.format(count)
    plt.text(n,count+0.1,strt,ha='center')

plt.xlabel('unit_of_wage' , fontweight = 'bold')
plt.xticks(rotation=0)
plt.show()

/var/folders/9f/00d3gq1s0w3_0m39lmyx76400000gn/T/
ipykernel_32543/1036700001.py:8: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    count = order1[n]
```



```
df.duplicated().sum()  
np.int64(0)
```

For FE:

1. Missing value - No
2. Categorical handler - Yes
3. Outlier - Yes
4. Imbalance - Yes
5. Duplicate - No
6. Drop any column - Yes
7. Scaling - Yes
8. Transformation - Yes

```
df.columns.tolist()  
['case_id',  
 'continent',  
 'education_of_employee',  
 'has_job_experience',  
 'requires_job_training',  
 'no_of_employees',  
 'yr_of_estab',  
 'region_of_employment',  
 'prevailing_wage',  
 'unit_of_wage',
```

```
'full_time_position',  
'case_status']
```